

Langage SQL

PC/PSI

Table des matières

1	Introduction	2
1.1	Notion de base	2
1.2	SGBD	2
1.3	SQL	2
1.4	Norme SQL	3
2	Modèle Entité / Association	3
2.1	Démarche de construction d'une base de données	3
2.2	Représentation graphique Entité / Association	3
2.3	identifiant	4
2.4	Cardinalité	5
3	Modèle de données relationnelles	6
3.1	Tables et données	6
3.2	Domaines	7
3.3	Les clés	7
3.4	passage du modèle E/A au modèle relationnel	8
4	Interrogation des données	9
4.1	Projection (éléments du SELECT)	10
4.1.1	Extraction de toutes les colonnes	10
4.1.2	Extraction de certaines colonnes	10
4.1.3	Alias	11
4.1.4	Duplicatas	11
4.1.5	Expressions et valeurs	12
4.1.6	Ordonnancement	12
4.1.7	Limitation du nombre de résultat	13
4.2	Restriction [WHERE]	13
4.2.1	Opérateurs de comparaison	14
4.2.2	Opérateurs logiques	14
4.3	Regroupements	15
4.3.1	Utilisation sans GROUP BY	15
4.3.2	Utilisation avec GROUP BY et/ou HAVING	16
4.4	opérateurs ensemblistes	17
4.4.1	Intersection	18
4.4.2	Union	18

4.4.3	Différence	19
4.4.4	Produit cartésien	20
4.5	Equi-jointure	22



Objectifs :

— Traduire une requête en langage SQL

1 Introduction

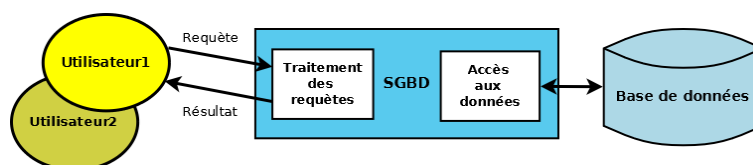
1.1 Notion de base

une base de données est un ensemble d'informations partagées par plusieurs utilisateurs. Ces informations sont interrogées et mises à jour par l'intermédiaire d'un logiciel. Une base de données est un ensemble structuré d'informations **non redondantes** dont l'organisation est régie par un modèle de données.

Les bases de données sont utilisées par de nombreuses entreprises dans toutes les industries. Elles sont notamment utilisées par les compagnies aériennes pour gérer les réservations. Elles sont utilisées pour la gestion de production.

1.2 SGBD

Le logiciel qui permet d'interagir avec une BD est un Système de Gestion de Base de Données (*SGBD*)



Il permet aux utilisateurs de créer et maintenir une base de données. Les activités supportées sont la définition d'une base de données (spécification des types de données à stocker), la construction d'une base de données (stockage des données proprement dites) et la manipulation des données (principalement ajouter, supprimer, retrouver des données).

1.3 SQL

SQL signifie « Structured Query Language » c'est-à-dire « Langage d'interrogation structuré ». En fait SQL est un langage complet de gestion de bases de données . Il a été conçu par IBM dans les années 70. Il est devenu le langage standard des systèmes de gestion de bases de données (SGBD).

1.4 Norme SQL

SQL a été normalisé dès 1986 mais les premières normes, trop incomplètes, ont été ignorées par les éditeurs de SGBD.

La norme SQL2 (appelée aussi SQL92) date de 1992. Le niveau « Entry Level » est à peu près respecté par tous les SGBD relationnels qui dominent actuellement le marché.

SQL-2 définit trois niveaux :

- « Full SQL » (ensemble de la norme)
- « Intermediate SQL »
- « Entry Level » (ensemble minimum à respecter pour se dire à la norme SQL-2)

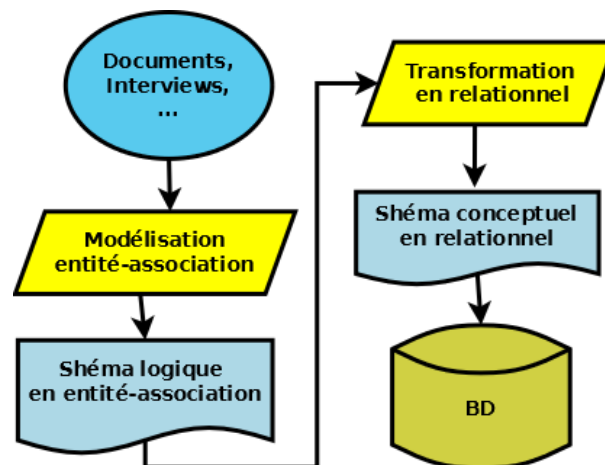
SQL3 (appelée aussi SQL99) est la nouvelle norme SQL. Malgré ces normes, il existe des différences non négligeables entre les syntaxes et fonctionnalités des différents SGBD. En conséquence, écrire du code portable n'est pas toujours simple dans le domaine des bases de données.

2 Modèle Entité / Association

2.1 Démarche de construction d'une base de données

Pour construire une base de données, il faut :

- construire un schéma conceptuel, modélisé sous forme d'entités et d'associations ;
- transformer le schéma E/A en schéma relationnel ;
- mettre en œuvre via un SGBD.

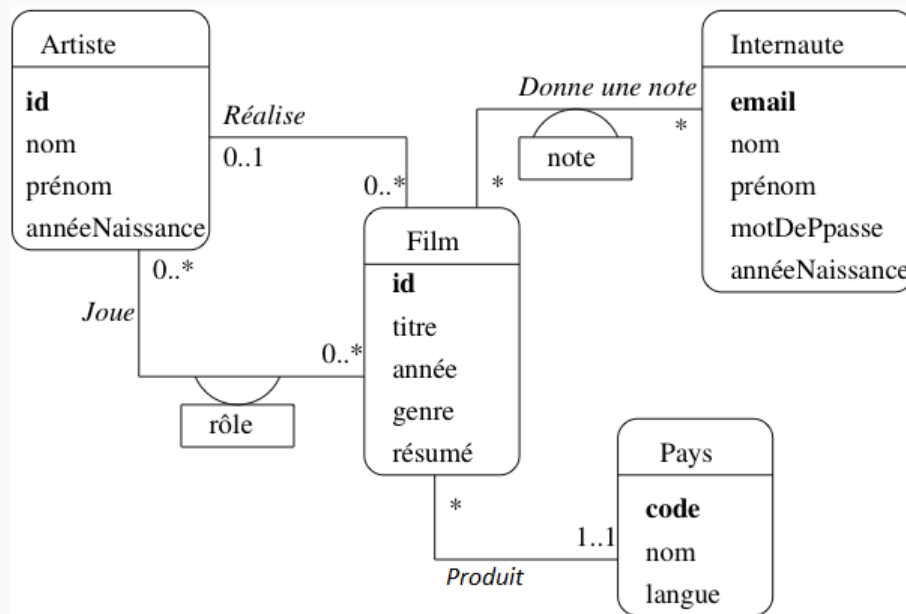


2.2 Représentation graphique Entité / Association

Le modèle Entité / Association propose une représentation graphique qui permet de concevoir puis de comprendre la structure d'une base de données. Dans ce modèle, on distingue les entités (éléments qui constituent la base de données) et les associations (c'est à dire les relations) entre ces entités

**Exemple :**

Nous prenons l'exemple d'une base de données décrivant des films, avec leur metteur en scène et leurs acteurs, ainsi que les cinémas où passent ces films. Nous supposons également que cette base de données est accessible sur le Web et que des internautes peuvent noter les films qu'ils ont vus.



- Les entités sont représentées par des rectangles, ici **Film**, **Artiste**, **Internaute** et **Pays**.
- Les associations entre entités sont représentées par des liens entre ces rectangles. Ici on a représenté par exemple le fait qu'un artiste joue dans des films, qu'un internaute note des films, etc.
- Chaque entité est caractérisée par un ensemble d'attributs, par exemple pour l'entité **Internaute**, son **email**, son **nom**, son **mot de passe** et son **année** de naissance.
- Une association peut également posséder des attributs comme l'attribut **note** affecté à l'association **donne une note**.

2.3 identifiant

**Définition : Identifiant**

Attribut ou groupe d'attributs dont la valeur identifie sans ambiguïté une occurrence d'une entité.

**Remarque :**

- L'identifiant est mis en relief (souvent soulignée ou en gras).
- On privilégie l'identifiant le plus court, le plus naturel (numéro bon de commande, numéro étudiant,...)
- On peut créer un identifiant artificiel par commodité.

2.4 Cardinalité

On peut remarquer sur l'exemple de la figure précédente les symboles $0..*$, $0..1$, $1..1$ et $*$. ils représentent la cardinalité de l'association, c'est-à-dire les nombres minimal (chiffre de gauche) et maximal (chiffre de droite) assignés à l'association entre les deux entités.



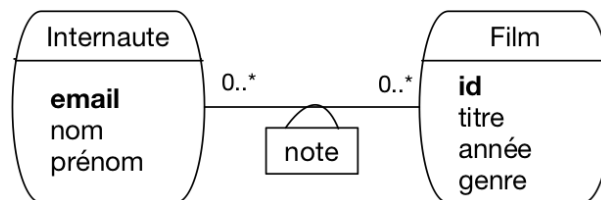
Remarque :

- La notation $*$ est équivalente à $0..*$;
- la notation 1 est équivalente à $1..1$.

Dans l'exemple précédent :

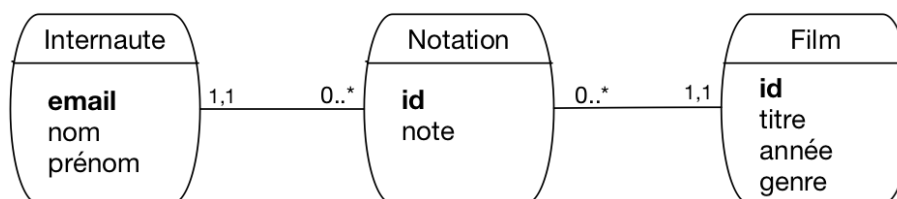
- pour l'association **Réalise** :
 - $0..*$: signifie qu'un artiste peut ne pas réaliser de film (0) et qu'il peut réaliser autant de films qu'il le souhaite ($*$) ;
 - $0..1$: (0) signifie qu'un film présent dans la base de données peut ne pas être associé avec un réalisateur (on s'autorise ici à ne pas le connaître) et qu'un film ne peut être réalisé que par un seul artiste (1).
- pour l'association **Produit** :
 - $*$: signifie qu'un pays peut ne pas produire de film (0) et qu'il peut produire autant de films qu'il le souhaite ($*$) ;
 - $1..1$: indique que l'on doit toujours connaître le pays producteur d'un film.

On souhaite dans notre exemple qu'un même **internaute** note plusieurs fois un même **film** si son opinion évolue.



Cela implique que chaque note de cet internaute devra être associée avec les noms, prénoms, e-mails de celui-ci qui seront alors stockés autant de fois qu'il y a de notes. Il y aura redondance des informations.

Pour lever cette difficulté, on peut transformer cette association $*..*$ en deux relations de cardinalités $0..*$ et $*..0$, le tout en introduisant une nouvelle entité.



Ici, les cardinalités minimales et maximales ont été fixées à 1 aux deux extrémités car une **note** ne saurait exister si un **internaute** ou un **film** ne sont pas précisés. Ainsi, l'association de cardinalité $0..*-0..*$ (aussi notée $*..*$) a été transformée en deux associations de cardinalités $1..1-0..*$ (aussi notées $1-*$).

3 Modèle de données relationnelles

Le modèle de données relationnel repose sur une théorie rigoureuse bien qu'adoptant des principes simples. La table relationnelle (relational table) est la structure de données de base qui contient des enregistrements appelés aussi « lignes » (tuple ou n-uplet). Une table est composée de colonnes (attributs) qui décrivent les enregistrements.

3.1 Tables et données

Considérons la figure suivante qui présente deux tables relationnelles permettant de stocker des compagnies, des pilotes et le fait qu'un pilote soit embauché par une compagnie :

Compagnie

comp	nrue	rue	ville	nomComp
AF	10	Gambetta	Paris	Air france
SING	7	Camparols	Singapour	Singapour AL
...	...	...	...	

Pilotes

brevet	nom	nbHvol	compa
PL1	Louise Ente	450	AF
PL2	Jules Ente	0	AF
PL3	Paul Vernes	1000	SING
...	...	...	...

Remarque : *Concepts*

- Attribut : nom donné à une colonne d'une table (exemple cote, Titre, Editeur, etc.). La première ligne de la table comporte les noms de ses attributs.
- Nom de la table (ou de la relation) : Compagnie, Pilotes ;
- Tuple (ou n-uplet) : nom donné à une ligne comportant des valeurs saisies. (tuple : AF ; 10 ; Gambetta ; Paris ; Air france).
- L'ordre des lignes et des colonnes n'est pas significatif
- Pas de lignes identiques
- Une case une valeur

Remarque : *Notation*

Pour présenter la définition d'une table relationnelle on utilise la notation du type : Compagnie(**comp**, nrue, rue, ville, nomComp) .
On peut aussi trouver cette même notation en précisant le domaine des attributs : Compagnie(**comp** : texte, nrue : entier, rue : texte, ville : texte, nomComp : texte)

3.2 Domaines

Pour passer du modèle à son implémentation dans le système de gestion de bases de données, pour chacun des attributs, on peut préciser son domaine (l'ensemble des valeurs que peut prendre l'attribut).

Par exemple, l'entité **Compagnie** contient les attributs :

- comp : texte,
- nrue : nombre entier,
- rue : texte,
- ville : texte,
- nomComp : texte;

3.3 Les clés



Définition : *Clef primaire.*

La clé primaire (primary key) d'une table est l'ensemble minimal de colonnes qui permet d'identifier de manière unique chaque enregistrement.

Dans les tables précédentes, les colonnes « clés primaires » sont notées en gras. La colonne **comp** représente le code de la compagnie et la colonne **brevet** décrit le numéro du brevet.



Définition : *Clef candidate.*

Une clé est dite « candidate » (candidate key) si elle peut se substituer à la clé primaire à tout instant. Une table peut contenir plusieurs clés candidates ou aucune.

Dans notre exemple, les colonnes **nomComp** et **nom** peuvent être des clés candidates si on suppose qu'aucun homonyme n'est permis.



Définition : *Clef étrangère.*

Une clé étrangère (foreign key) référence dans la majorité des cas une clé primaire d'une autre table (sinon une clé candidate sur laquelle un index unique aura été défini). Une clé étrangère est composée d'une ou de plusieurs colonnes. Une table peut contenir plusieurs clés étrangères ou aucune.

Dans notre exemple, la colonne **compa** (notée en italique dans les tables) est une clé étrangère, car elle permet de référencer un enregistrement unique de la table **Compagnie** via la clé primaire **comp**. Le modèle relationnel est ainsi fondamentalement basé sur les valeurs. Les associations entre tables sont toujours binaires et assurées par les clés étrangères. Les clés primaires et étrangères seront définies dans les tables en SQL à l'aide de contraintes.

**Remarque : Notation**

Pour présenter la définition d'une table relationnelle on utilise la notation du type : Compagnie(**comp**, nrue, rue, ville, nomComp) .

On peut aussi trouver cette même notation en précisant le domaine des attributs : Compagnie(**comp** : texte, nrue : entier, rue : texte, ville : texte, nomComp : texte)

3.4 passage du modèle E/A au modèle relationnel

Il existe des règles permettant de passer du modèle E/A au modèle relationnel. Ces règles ne sont **pas au programme de CPGE** mais aident à comprendre les interdépendances entre les différentes tables.

Règle 1 :

Chaque entité se transforme en une table relationnelle et ses attributs vont constituer les colonnes de la table.

Règle 2 : association binaire $x..1 - x..*$

On parle ici des cardinalités $0..1 - 0..*$, $0..1 - 1..*$, $1..1 - 0..*$ et $1..1 - 1..*$.

La clef primaire issue de l'entité coté cardinalité $x..1$ est dupliquée dans la table issue de l'entité coté cardinalité $x..*$ où elle devient clef étrangère.

Règle 3 : association binaire $0..1 - 1..1$

On applique la même règle.

La clef primaire issue de l'entité coté cardinalité $1..1$ est dupliquée dans la table issue de l'entité coté cardinalité $0..1$ où elle devient clef étrangère.

Règle 4 : association binaire $0..1 - 0..1$

La clef primaire issue de la table d'une des entités est dupliquée dans la table issue de l'autre entité où elle devient clef étrangère.

Règle 5 : association binaire $1..1 - 1..1$

Ce cas est rarement rencontré mais dans ce cas on fusionne les deux tables issues des entités de l'association.

Règle 6 : association binaire $x..* - x..*$

On parle ici des cardinalités $0..* - 0..*$, $0..* - 1..*$, $1..* - 0..*$ et $1..* - 1..*$. On crée une table supplémentaire ayant comme clef primaire les clefs primaires des deux tables. Si l'association contient elle même des attributs, ceux-ci deviennent attribut de la table supplémentaire.

Exercice

Réaliser le modèle relationnel de l'exemple de la page 4.

4 Interrogation des données

Ce chapitre traite de l'aspect le plus connu du langage SQL qui concerne l'extraction des données par requêtes (nom donné aux instructions **SELECT**). Une requête permet de rechercher des données dans une ou plusieurs tables ou vues, à partir de critères simples ou complexes.

**Remarque :**

Il est préférable d'utiliser les conventions suivantes :

- Tous les mots-clés de SQL sont notés en majuscules.
- Les noms de tables sont notés en Minuscules (excepté la première lettre).
- Les noms de colonnes et de contraintes en minuscules.

L'adoption de ces conventions rendra vos requêtes, scripts et programmes plus lisible

La figure suivante schématise les principales fonctionnalités de l'instruction **SELECT**. Celle-ci est composée d'une directive **FROM** qui précise la (les) table(s) interrogée(s), et d'une directive **WHERE** qui contient les critères.

**Principe : Syntaxe générale du *SELECT***

La syntaxe SQL simplifiée de l'instruction **SELECT** est la suivante :

```
SELECT [ { DISTINCT | DISTINCTROW } | ALL ] listeColonnes
FROM nomTable1[,nomTable2]...
[ WHERE condition ]
[ clauseRegroupement ]
[ HAVING condition ]
[ clauseOrdonnancement ]
[ LIMIT [décalage,] nbLignes ] ;
```

Nous détaillerons chaque option à l'aide d'exemples au cours de ce chapitre.

4.1 Projection (éléments du SELECT)



Principe : *Projection*

Syntaxe (Les commandes entre [] sont optionnelles) :

```
SELECT [ DISTINCT | ALL ] listeColonnes
FROM nomTable [aliasTable]
[ clauseOrdonnancement ]
[ LIMIT [ rangDépart ,] nbLignes ] ;
```

- DISTINCT joue le même rôle, à savoir ne pas inclure les duplicatas.
- ALL prend en compte les duplicatas (option par défaut).
- listeColonnes : * | expression1 [[AS] alias1] [,expression2 [[AS] alias2]...
- * : extrait toutes les colonnes de la table.
- expression : nom de colonne, fonction SQL, constante ou calcul.
- alias : renomme l'expression (nom valable pendant la durée de la requête).
- FROM désigne la table (qui porte un alias ou non) à interroger.
- clauseOrdonnancement : tri sur une ou plusieurs colonnes ou expressions.
- LIMIT pour limiter le nombre de lignes après résultat.

4.1.1 Extraction de toutes les colonnes

L'extraction de toutes les colonnes d'une table requiert l'utilisation du symbole « * »



Code

```
SELECT * FROM Pilote ;
```

brevet	nom	nbHvol	compa
PL1	Louise Ente	450	AF
PL2	Jules Ente	0	AF
PL3	Paul Vernes	1000	SING
PL4	Placide Fresnay	2450	CAST
PL5	Daniel Viel	NULL	AF
...	...	...	...

4.1.2 Extraction de certaines colonnes



Code

```
SELECT compa, brevet FROM Pilote ;
```

compa	brevet
AF	PL1
AF	PL2
SING	PL3
CAST	PL4
AF	PL5
...	...

4.1.3 Alias

Les alias permettent de renommer des colonnes à l’affichage ou des tables dans la requête. Les alias de colonnes sont utiles pour les calculs.



Remarque :

L’utilisation de la directive AS est facultative mais il est préférable de l’utiliser.



Code

```
SELECT compa AS c1,
        nom AS NometPrenom,
        brevet c3
FROM Pilote;
```



Code

```
SELECT aliasPilotes.compa AS c1,
        aliasPilotes.nom
FROM Pilote aliasPilotes;
```

c1	NometPrenom	c3
AF	Louise Ente	PL1
AF	Jules Ente	PL2
SING	Paul Vernes	PL3
CAST	Placide Fresnay	PL4
AF	Daniel Viel	PL5
...	...	...

c1	nom
AF	Louise Ente
AF	Jules Ente
SING	Paul Vernes
CAST	Placide Fresnay
AF	Daniel Viel
...	...

4.1.4 Duplicatas

La directive DISTINCT élimine les éventuels duplicatas.



Code

```
SELECT DISTINCT compa FROM Pilote;
```

compa
AF
SING
CAST

4.1.5 Expressions et valeurs

Il est possible d'évaluer et d'afficher simultanément des expressions dans la clause SELECT (types numériques, DATE et DATETIME). Les opérateurs arithmétiques sont évalués par ordre de priorité (*, /, + et -). Le résultat d'une expression comportant une valeur NULL est évalué à NULL

Code

```
SELECT brevet , nbHVol ,
        nbHVol*nbHVol AS auCarre ,
        10*nbHVol+5/2
FROM Pilote ;
```

brevet	nbHvol	auCarre	10*nbHVol+5/2
PL1	450	202500	4502.5
PL2	0	0	0
PL3	1000	1000000	10002.5
PL4	2450	6002500	24502.5
PL5	NULL	NULL	NULL
...	...	...	...

4.1.6 Ordonnancement

Pour trier le résultat d'une requête, il faut spécifier la clause d'ordonnancement par ORDER BY de la manière suivante :

```
ORDER BY
{ expression1 | alias1 } [ASC | DESC]
[, {expr2 | alias2} [ASC | DESC]
```

- expression : nom de colonne, fonction SQL, constante, calcul.
- ASC ou DESC : tri ascendant ou descendant (par défaut ASC).

Code

```
SELECT brevet ,nbHVol
FROM Pilote ORDER BY nbHvol DESC ;
```

brevet	nbHvol
PL4	2450
PL3	1000
PL1	450
PL2	0
PL5	NULL

4.1.7 Limitation du nombre de résultat

La clause LIMIT est à utiliser pour spécifier le nombre maximum de résultats que l'on souhaite obtenir. Cette clause est souvent associée à un décalage sur le jeu de résultat.

LIMIT [décalage,] nbLignes]

ou bien

LIMIT nbLignes OFFSET décalage



Code

```
SELECT brevet ,nbHVol
      FROM Pilote ORDER BY nbHvol DESC LIMIT 3,1;
```

ou



Code

```
SELECT brevet ,nbHVol
      FROM Pilote ORDER BY nbHvol DESC LIMIT 1 OFFSET 3;
```

brevet	nbHvol
PL3	1000

4.2 Restriction [WHERE]



Principe :

Les éléments de la clause WHERE d'une requête permettent de programmer l'opérateur de restriction. Cette clause limite la recherche aux enregistrements qui respectent une condition simple ou complexe.

```
SELECT [ DISTINCT | ALL ] listeColonnes
FROM nomTable [aliasTable]
[ WHERE condition ] ;
```

- condition composée de colonnes, d'expressions, de constantes liées deux à deux entre des opérateurs :
 - de comparaison (>, =, <, >=, <=, <>) ;
 - logiques (NOT , AND ou OR) ;

4.2.1 Opérateurs de comparaison



Code

```
SELECT brevet , nom "de_Air-France"
FROM Pilote WHERE compa = 'AF';
```

brevet	de Air-France
PL1	Louise Ente
PL2	Jules Ente
PL5	Daniel Viel
...	...



Code

```
SELECT brevet , nom, nbHvol
FROM Pilote WHERE nbHvol<500;
```

brevet	nom	nbHvol
PL1	Louise Ente	450
PL2	Jules Ente	0
...	...	...

4.2.2 Opérateurs logiques

- L'ordre de priorité des opérateurs logiques est NOT , AND et OR .
- Les opérateurs de comparaison (>, =, <, >=, <=, <>) sont prioritaires par rapport à NOT .
- Les parenthèses permettent de modifier ces règles de priorité.



Code

```
SELECT brevet , nom, compa
FROM Pilote WHERE
(compa = 'SING' OR compa = 'AF' AND nbHVol < 500);
```

brevet	nom	compa
PL1	Louise Ente	AF
PL2	Jules Ente	AF
PL3	Paul Vernes	SING
...	...	...

4.3 Regroupements



Principe :

Cette section traite à la fois des regroupements de lignes (agrégats) et des fonctions de groupe (ou multilignes).

```
SELECT [ DISTINCT | ALL ] listeColonnes
FROM nomTable [ WHERE condition ]
[ clauseRegroupement ]
[ HAVING condition ]
[ clauseOrdonnement ]
[ LIMIT [décalage,] nbLignes ] ;
```

- `listeColonnes` : peut inclure des expressions (présentes dans la clause de regroupement) ou des fonctions de groupe.
- `clauseRegroupement` : `GROUP BY (expression1[,expression2]...)` permet de regrouper des lignes selon la valeur des expressions (colonnes, fonction, constante, calcul).
- `HAVING condition` : pour inclure ou exclure des lignes aux groupes (la condition ne peut faire intervenir que des expressions du `GROUP BY`).
- `ClauseOrdonnement` : déjà étudié (`ORDER BY` dans la section Projection/Ordonnement).

Voici les fonctions usuelles :

Fonction	Objectif
<code>AVG([DISTINCT] expr)</code>	Moyenne de <code>expr</code> (nombre).
<code>COUNT(* [DISTINCT] expr)</code>	Nombre de lignes (* toutes les lignes, <code>expr</code> pour les colonnes non nulles).
<code>MAX([DISTINCT] expr)</code>	Maximum de <code>expr</code> (nombre, date, chaîne).
<code>MIN([DISTINCT] expr)</code>	Minimum de <code>expr</code> (nombre, date, chaîne).
<code>SUM([DISTINCT] expr)</code>	Somme de <code>expr</code> (nombre).

Soit une nouvelle table Pilotes :

Pilotes						
brevet	nom	nbHVol	prime	embauche	typeAvion	compa
PL-1	Gratien Viel	450	500	1965-02-05	A320	AF
PL-2	Didier Donsez	0		1995-05-13	A320	AF
PL-3	Richard Grin	1000		2001-09-11	A320	SING
PL-4	Placide Fresnais	2450	500	2001-09-21	A330	SING
PL-5	Daniel Vielle	400	600	1965-01-16	A340	AF
PL-6	Francoise Tort		0	2000-12-24	A340	CAST

4.3.1 Utilisation sans GROUP BY

Utilisées sans `GROUP BY`, ces fonctions s'appliquent à la totalité ou à une seule partie d'une table comme le montrent les exemples suivants.

Moyenne des heures de vol et des primes des pilotes de la compagnie 'AF'.

 Code

```
SELECT AVG(nbHvol) ,AVG(prime)
FROM Pilote WHERE compa ='AF' ;
```

AVG(ndHvol)	AVG(prime)
283.333	550.000

Nombre de pilotes, d'heures de vol et de primes (toutes et distinctes) recensées dans la table.

 Code

```
SELECT COUNT(*) ,COUNT(nbHvol) ,
COUNT(prime) , COUNT(DISTINCT prime)
FROM Pilote ;
```

COUNT(*)	COUNT(nbHvol)	COUNT(prime)	COUNT(DISTINCT prime)
6	5	4	3

4.3.2 Utilisation avec GROUP BY et/ou HAVING

- La clause WHERE de la requête permet d'exclure des lignes pour chaque groupement, ou de rejeter des groupements entiers. Elle s'applique donc à la totalité de la table.
- La clause GROUP BY liste les colonnes du groupement.
- La clause HAVING permet de poser des conditions sur chaque groupement.
- La clause ORDER BY permet de trier le résultat (déjà étudiée).

 Remarque :

Les colonnes présentes dans le SELECT doivent apparaître dans le GROUP BY. Seules des fonctions ou expressions peuvent exister en plus dans le SELECT. Les alias de colonnes ne peuvent pas être utilisés dans la clause GROUP BY.

Dans le premier exemple suivant, en groupant sur la colonne compa , trois ensembles de lignes (groupements) sont composés. Il est alors possible d'appliquer des fonctions de groupe à chacun de ces ensembles

Moyenne des heures de vol et des primes pour chaque compagnie.

Nombre de pilotes qualifiés par type d'appareil et par compagnie.

Code

```
SELECT compa, AVG(nbHVol),
AVG(prime) FROM Pilote
GROUP BY(compa);
```

compa	AVG(nbHVol)	AVG(prime)
AF	283.333	550.000
CAST	NULL	0.000
SING	1725.000	500.000

Code

```
SELECT compa, typeAvion,
COUNT(brevet) FROM Pilote
GROUP BY compa, typeAvion;
```

compa	typeAvion	COUNT(brevet)
AF	A320	2
AF	A340	1
CAST	A340	1
SING	A320	1
SING	A330	1

Compagnies (et nombre de leurs pilotes) ayant plus d'un pilote.

Code

```
SELECT compa, COUNT(brevet) FROM Pilote
GROUP BY compa HAVING COUNT(brevet)>=2 ;
```

compa	COUNT(brevet)
AF	3
SING	2

4.4 opérateurs ensemblistes

- Le langage SQL permet de programmer les opérations binaires (entre deux tables) suivantes :
- intersection qui extrait des données présentes simultanément dans les deux tables ;
 - union par les opérateurs UNION et UNION ALL qui fusionnent des données des deux tables ;
 - différence qui extrait des données présentes dans une table sans être présentes dans la deuxième table ;
 - produit cartésien par le fait de disposer de deux tables dans la clause FROM , ce qui permet de composer des combinaisons à partir des données des deux tables.

Remarque :

Seules des colonnes de même type (CHAR , VARCHAR , DATE , numériques, etc.) doivent être comparées avec des opérateurs ensemblistes.

Étudions à présent chaque opérateur à partir de l'exemple composé des deux tables suivantes :

- AvionsdeAF(immat CHAR(6), typeAvion CHAR(10), nbHVol DECIMAL(10,2))
- AvionsdeSING(immatriculation CHAR(6), typeAv CHAR(10), prixAchat DECIMAL(14,2))

AviondeAF		
immat	typeAvion	nbHVol
F-WTSS	Concorde	6570
F-GLFS	A320	3500
F-GTMP	A340	

AviondeSING		
immatriculation	typeAvion	prixAchat
S-ANSI	A320	104 500
S-AVEZ	A320	156 000
S-SMILE	A330	198 000
F-GTMP	A340	204 500

4.4.1 Intersection

L'opérateur INTERSECT élimine les duplicatas avant d'opérer l'intersection.



Principe :

L'intersection entre deux ensembles homogènes se programme à l'aide d'une requête du type :

```
SELECT DISTINCT ensemble1 FROM Table1
INTERSECT
SELECT ensemble2 FROM Table2
```

Comme l'opérateur d'intersection, cette requête est commutative (Table1 peut jouer le rôle de Table2, et ensemble1 celui d'ensemble2).

A l'affichage le nom des colonnes est donné par la première requête. La deuxième fait apparaître deux colonnes dans le SELECT

Quels sont les types d'avions que les deux compagnies exploitent en commun ?

Quels sont les avions qui sont exploités par les deux compagnies en commun ?



Code

```
SELECT typeAvion FROM AvionsdeAF
INTERSECT
(SELECT typeAv FROM AvionsdeSING);
```

typeAvion
A320
A340



Code

```
SELECT immatriculation ,typeAv FROM AvionsdeSING
INTERSECT
SELECT immat ,typeAvion FROM AvionsdeAF ;
```

immatriculation	typeAv
F-GTMP	A340

4.4.2 Union



Principe :

Les opérateurs **UNION** et **UNION ALL** sont commutatifs. L'opérateur UNION permet d'éviter les duplicatas (comme DISTINCT dans un SELECT). UNION ALL ne les élimine pas.

Quels sont tous les types d'avions que les deux compagnies exploitent ?

Même requête avec les duplicatas.

Code

```
SELECT typeAvion FROM AvionsdeAF
UNION
SELECT typeAv FROM AvionsdeSING ;
```

typeAvion
A320
A340
Concorde
A330

Code

```
SELECT typeAvion FROM AvionsdeAF
UNION ALL
SELECT typeAv FROM AvionsdeSING ;
```

typeAvion
A320
A340
Concorde
A340
A320
A320
A330

Remarque :

L'intersection entre deux ensembles homogènes peut également se programmer à l'aide d'une requête du type :

```
SELECT DISTINCT ensemble1 FROM Table1 WHERE ensemble1
IN (SELECT ensemble2 FROM Table2)
```

4.4.3 Différence

L'opérateur EXCEPT est le seul opérateur ensembliste qui ne soit pas commutatif. Il élimine les duplicatas avant d'opérer la soustraction.

Principe :

La différence entre deux ensembles homogènes se programme à l'aide d'une requête du type :

```
SELECT ensemble1 FROM Table1
EXCEPT
SELECT ensemble2 FROM Table2.
```

Quels sont les types d'avions exploités par la compagnie 'AF', mais pas par 'SING' ?

Quels sont les types d'avions exploités par la compagnie 'SING,' mais pas par 'AF' ?

Code

```
SELECT typeAvion FROM AvionsdeAF
EXCEPT
SELECT typeAv FROM AvionsdeSING ;
```

typeAvion

Concorde

Code

```
SELECT typeAv FROM AvionsdeSING
EXCEPT
SELECT typeAvion FROM AvionsdeAF ;
```

typeAvion

A330

Remarque :

La différence entre deux ensembles homogènes peut également se programmer à l'aide d'une requête du type :

```
SELECT DISTINCT ensemble1 FROM Table1 WHERE ensemble1
NOT IN (SELECT ensemble2 FROM Table2).
```

Quels sont les types d'avions exploités par la compagnie 'AF', mais pas par 'SING' ?

Code

```
SELECT typeAvion FROM AvionsdeAF
WHERE typeAvion NOT IN
(SELECT typeAv FROM AvionsdeSING) ;
```

4.4.4 Produit cartésien

Principe :

Le produit cartésien de deux tables T1 et T2 est l'ensemble des enregistrements (x, y) où $x \in T1$ et $y \in T2$. Le produit cartésien total entre deux tables T1 et T2 se programme sous SQL en positionnant les deux tables dans la clause **FROM**, sans ajouter de conditions dans la clause **WHERE**.

Soit les deux tables suivantes :

Pilotes

brevet	nom	nbHVol	compa
PL-1	Gratien Viel	450	AF
PL-2	Richard Grin	1000	SING
PL-3	Placide Fresnais	2450	CAST
PL-4	Daniel Vielle	5000	AF

AviondeAF

immat	typeAvion	nbHVol
F-WTSS	Concorde	6570
F-GLFS	A320	3500
F-GTMP	A340	



```
SELECT * FROM Pilote , AvionsdeAF ;
```

brevet	nom	nbHVol	compa	immat	typeAvion	nbHVol
PL-1	Gratien Viel	450	AF	F-WTSS	Concorde	6570
PL-1	Gratien Viel	450	AF	F-GLFS	A320	3500
PL-1	Gratien Viel	450	AF	F-GTMP	A340	
PL-2	Richard Grin	1000	SING	F-WTSS	Concorde	6570
PL-2	Richard Grin	1000	SING	F-GLFS	A320	3500
PL-2	Richard Grin	1000	SING	F-GTMP	A340	
PL-3	Placide Fresnais	2450	CAST	F-WTSS	Concorde	6570
PL-3	Placide Fresnais	2450	CAST	F-GLFS	A320	3500
PL-3	Placide Fresnais	2450	CAST	F-GTMP	A340	
PL-4	Daniel Vielle	5000	AF	F-WTSS	Concorde	6570
PL-4	Daniel Vielle	5000	AF	F-GLFS	A320	3500
PL-4	Daniel Vielle	5000	AF	F-GTMP	A340	

Quels sont les couples possibles (avion, pilote) en considérant les avions et les pilotes de la compagnie 'AF' ?



```
SELECT p.brevet , avAF.immat
FROM Pilote AS p, AvionsdeAF AS avAF
WHERE p.compas = 'AF' ;
```

brevet	immat
PL-1	F-GLFS
PL-1	F-GTMP
PL-1	F-WTSS
PL-4	F-GLFS
PL-4	F-GTMP
PL-4	F-WTSS

4.5 Equi-jointure



Principe :

Une équi-jointure met en relation deux tables sur la base d'une clause de jointure (comparaison de colonnes). Généralement, cette comparaison fait intervenir une clé étrangère d'une table avec une clé primaire d'une autre table.

```
SELECT [DISTINCT] listeColonnes
FROM [nomBase.] nomTable1
JOIN [nomBase.] nomTable2 ON condition
[ WHERE condition ] ;
```

Soit les deux tables suivantes :

Pilotes

brevet	nom	nbHVol	compa
PL-1	Pierre Lamothe	450	AF
PL-2	Didier Linxe	900	AF
PL-3	Christian Soutou	1000	SING
PL-4	Henri Alquié	3400	AF

Compagnie

comp	nrue	rue	ville	nomComp
AF	10	Gambetta	Paris	Air france
SING	7	Camparols	Singapour	Singapour AL
CAST	1	G. Brassens	Blagnac	Castanet AL



Code

```
SELECT *
FROM Compagnie
JOIN Pilote ON comp = compa
```

brevet	nom	nbHVol	compa	nrue	rue	ville	nomComp
PL-1	Pierre Lamothe	450	AF	10	Gambetta	Paris	Air france
PL-2	Didier Linxe	900	AF	10	Gambetta	Paris	Air france
PL-3	Christian Soutou	1000	SING	7	Camparols	Singapour	Singapour AL

Extrayons l'identité des pilotes de la compagnie de nom 'Air France' ayant plus de 500 heures de vol :

Code

```
SELECT brevet , nom
FROM Compagnie
JOIN Pilote ON comp = compa
WHERE nomComp = 'Air_France'
AND nbHVol > 500 ;
```

brevet	nom
PL-2	Didier Linxe
PL-4	Henri Alquié

Extrayons les coordonnées des compagnies qui embauchent des pilotes de moins de 500 heures de vol :

Code

```
SELECT nomComp, nrue , rue , ville
FROM Compagnie
JOIN Pilote ON comp = compa
WHERE nbHVol < 500 ;
```

nomComp	nrue	rue	ville
Air France	124	Port Royal	Paris